Proiect – Coduri Liniare

Stancu Catalin-Gabriel – Grupa 311

February 22, 2026

Contents

1 Abstract **2**
1.1 Care e problema pe care incercam sa o rezolvam 2
1.2 Solutia propusa de noi 2
1.3 Ce rezultate s-au obtinut 2

2 Introducere **2**
2.1 Problema generala 2
2.2 Motivatie 3
2.3 Aplicatii 3

3 State of the art **3**
3.1 Detalii matematice si explicatii 3
3.1.1 Coduri 3
3.1.2 Coduri Liniare 5
3.1.3 Matrici de Parity check 7
3.1.4 Codul Hamming 8
3.1.5 Dualul unui Cod liniar 10
3.1.6 Codul Hadamard 10

4 Algoritm si implementare **10**

5 Experimente **12**
5.1 Plot-uri cu performanta 12

6 Concluzii si future work **13**
6.1 Cateva concluzii 13
6.2 Future work 14

1 Abstract

1.1 Care e problema pe care incercam sa o rezolvam

Avem un mesaj m constand din k simboluri din multimea finita Σ cu $|\Sigma| = n$ (pe care o vom numi alfabet), pe care vrem sa-l transmitem cuiva, printr-un canal de comunicatie zgomotos. Deci mesajul care ajunge la receptor poate fi alterat de catre zgomot.

Vrem sa putem codifica mesajul inainte de a-l trimite, asa incat, din versiunea sa alterata de zgomot care ajunge la receptor, acesta sa poata corecta erorile si recupera mesajul original m.

1.2 Solutia propusa de noi

Vom defini o clasa de modalitati de a codifica mesajul m, adica o clasa de coduri – coduri liniare, ce se bazeaza pe structura de spatiu vectorial a multimii tuturor cuvintelor cu n litere pentru a oferi o modalitate de a recupera mesajul original. Apoi vom defini codurile Hamming si codurile Hadamard, si vom vedea si compara parametrii de interes ai lor.

1.3 Ce rezultate s-au obtinut

Codul Hamming generalizat este bun pentru a evita redundanta, dar nu poate corecta si detecta decat un numar limitat de erori, in timp ce codul Hadamard generalizat poate corecta oricate erori, dar cu costul redundantei din ce in ce mai mari.

2 Introducere

2.1 Problema generala

Avem un emitator si un receptor. Emitatorul vrea sa transmita receptorului un mesaj care consta in un sir de k simboluri ce apartin multimii Σ pe care o vom numi alfabet. Presupunem ca Σ este corp finit $\Sigma = \mathbb{F}_q^k$ (unde q este putere de numar prim).

Transmiterea mesajului se face prin un canal de comunicare zgomotos, i.e. daca mesajul trimis este $m \in \mathbb{F}_q^k$, mesajul primit va fi $m + \varepsilon \in \mathbb{F}_q^k$ unde $\varepsilon \in \mathbb{F}_q^k$ nu este neaparat egal cu $0_{\mathbb{F}_q^k}$. Deci receptorul poate primi un mesaj diferit de cel pe care emitatorul a vrut sa i-l transmita.

Pentru a rezolva aceasta problema, asociem mesajului original $m \in \mathbb{F}_q^k$ un alt mesaj $m' \in \mathbb{F}_q^n$, eventual mai lung ($n \geq k$), ce contine componente redundante, dar cu ajutorul carora receptorul sa poata corecta erorile introduse de catre zgomot si recupera mesajul original m. Apoi trimitem catre receptor acest nou mesaj m'.

2.2 Motivatie

Problema apare oriunde este nevoie sa trimitem un mesaj catre un receptor printr-un canal de comunicatie zgomotos i.e. mesajul pe care il primeste receptorul poate sa difere de mesajul original pe care l-am trimis noi, din cauza zgomotului.

Aceasta problema apare si cand vrem sa stocam ceva in memoria calculatorului – eu transmit mesajul catre mine insumi din viitor, iar bitii stocati in calculator se pot modifica de-a lungul timpului din cauza unor fenomene fizice (de exemplu: temperatura mare, fluctuatii de tensiune, raze cosmice).

De exemplu: stocam un sir de biti pe un CD pentru a-i accesa in viitor, dar intre timp CD-ul se zgarie si deci bitii sunt schimbati i.e. zgomot, sau Voyager 1 si transmiterea de mesaje catre Pamant prin intermediul canalului zgomotos al spatiului.

2.3 Aplicatii

Codurile Hamming au fost folosite in memorii RAM ECC (error-correcting code memory), si in general in situatii unde erori de un singur bit sunt probabile.

Codurile Hadamard au fost folosite de catre NASA pentru a corecta erorile ce ar fi aparut in transmiterea catre Pamant a imaginilor de pe Marte, de catre nava spatiala Mariner 9[1].

3 State of the art

3.1 Detalii matematice si explicatii

Toate definitiile si majoritatea continutului din aceasta sectiune provine din [2], contributia mea fiind unele observatii, explicatii sau justificari.

3.1.1 Coduri

Definitie 1 (Distanta Hamming): *Fie Σ alfabet (multime) finit. Numim distanta Hamming functia $\Delta(.,.) : \Sigma^n \times \Sigma^n \to \mathbb{N}$ definita prin:*

$$\Delta(v, w) \overset{def.}{:=} |\{i | v_i \neq w_i\}|, \forall v, w \in \Sigma^n$$

unde $v = \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix}$ si $w = \begin{pmatrix} w_1 \\ \vdots \\ v_n \end{pmatrix}$, adica numarul de componente ale v si w in care acestia difera.

*Numim **distanta relativa Hamming** functia $\delta : \Sigma^n \times \Sigma^n \to [0, \infty)$ definita prin:*

$$\delta(v, w) \overset{def.}{:=} \frac{\Delta(v, w)}{n}$$

Numim ***bila Hamming centrata in*** v ***de raza*** r *multimea*

$$B_{Ham}(v,r) \overset{not.}{:=} \{w \in \Sigma^n | \Delta(v,w) \leq r\}$$

Se verifica usor ca distanta Hamming si distanta relativa Hamming sunt metrici pe Σ^n.

Definitie 2 (Ponderea Hamming): *Fie* Σ *alfabet (multime). Numim ponderea Hamming a lui* v *functia* $wt : \Sigma^n \to \mathbb{N}$ *definita prin:*

$$wt(v) \overset{def.}{:=} |\{v_i | v_i \neq 0_{\Sigma^n}\}|, \forall v \in \Sigma^n$$

unde $v = \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix}$, *adica numarul de componente ale* v *diferite de* 0_Σ.

Definitie 3 (Cod): *Numim Cod q-ar de corectare a erorilor de lungime* n *peste un alfabet finit* Σ *o submultime* C *a* Σ^n. *Elementele multimii* C *se vor numi codewords.*

Daca $|\Sigma| \overset{not.}{=:} q$ *atunci spunem ca* C *este un cod q-ar. In particular, daca* $q = 2$ *spunem ca* C *este un cod binar.*

Fie $\mathcal{M}$ *multimea mesajelor posibile. Asociata unui cod este si o functie injectiva* $E : \mathcal{M} \overset{inj.}{\to} \Sigma^n$ *cu* $Im(E) = C$, *pe care o numim* ***functie de codificare****.*

Definitie 4 (Rata unui Cod): *Fie* C *cod de lungime* n *peste* Σ. *Numim rata lui* C*:*

$$R(C) \overset{def.}{:=} \frac{\log(|C|)}{n \log(|\Sigma|)}$$

Definim ***dimensiunea codului*** C *ca fiind* $\frac{\log(|C|)}{\log(|\Sigma|)}$.

Observatie: $R(C) \overset{def.}{:=} \frac{\log(|C|)}{n\log(|\Sigma|)} = \frac{\log(\frac{1}{|C|})}{\log(\frac{1}{|\Sigma^n|})} = \frac{I_{\omega_{v,C}}}{I_{\omega_{v,\Sigma^n}}}$ unde $v \in \Sigma^n$ fixat, $\omega_{v,C}$ este evenimentul probabilist $\{v\} \in \mathcal{P}(C)$ iar ω_{v,Σ^n} este evenimentul probabilist $\{v\} \in \mathcal{P}(\Sigma^n)$, iar I_ω este informatia Shannon a evenimentului ω.

Rata unui cod masoara redundanta.

Definitie 5 (Distanta minima a unui Cod): *Fie* C *cod de lungime* n *peste* Σ. *Numim distanta minima a lui* C *si notam:*

$$\Delta(C) \overset{def}{:=} \min\{\Delta(v,w) | v, w \in C\}$$

Numim ***distanta relativa minima a lui*** C *si notam* $\delta(C)$ *cantitatea normalizata:*

$$\delta(C) \overset{def.}{:=} \frac{\Delta(C)}{n}$$

Urmatoarea propozitie arata cum notiunea de distanta minima a unui cod este folositoare pentru detectarea si corectarea erorilor.

Propozitie 6 (Distanta unui Cod si detectarea erorilor): *Fie C cod de lungime n peste Σ, cu distanta minima $\Delta(C) =: 2t+1$.*

Atunci:

(1) C poate fi folosit pentru a detecta orice eroare care modifica $2t$ componente.

(2) C poate fi folosit pentru a corecta orice eroare care modifica t componente.

(3) C poate fi folosit pentru a corecta orice stergeri de $2t$ componente.

Justificare: Avem $\Delta(C) = 2t+1$, deci $\forall v_1, v_2 \in C, \Delta(v_1, v_2) \leq 2t+1$. Deci pentru orice $c \in C$, bila Hamming $B_{Ham}(c, 2t)$ contine doar un singur element, anume centrul sau c.

Daca v' este versiunea alterata de zgomot a unui codeword $v \in C$, care difera de v in $s \leq 2t$ componente adica $\Delta(v, v') =: s \leq 2t$, atunci:

$$B_{Ham}(v, 2t) \ni v' \text{ si } v' \notin B_{Ham}(w, 2t) \forall w \in C \setminus \{v\}$$

si detectam astfel ca $v' \notin C$.

Daca in plus $\Delta(v, v') \leq t$ atunci:

$$B_{Ham}(v, t) \ni v \text{ si } v \notin B_{Ham}(w, t) \forall w \in C \setminus \{v\}$$

deci cel mai apropiat codeword de v' este v, deci stim ca v' a fost v inainte de alterarea sa de catre zgomot.

3.1.2 Coduri Liniare

Definitie 7 (Cod liniar): *Fie C cod q-ar de lungime n peste alfabetul finit Σ. Daca Σ este un corp (deci $\Sigma = \mathbb{F}_q$ unde q este putere de numar prim) iar C este un subspatiu vectorial al $\mathbb{F}_q$-spatiului vectorial $\Sigma^n = \mathbb{F}_q^n$, atunci spunem ca C este un cod liniar.*

Observatie: Daca $C \leq_{\mathbb{F}_q} \mathbb{F}_q^n$ este un cod liniar, din definitia ratei unui cod arbitrar avem:

$$R(C) \overset{def}{:=} \frac{\log(|C|)}{\log(|\Sigma^n|)} \overset{C subsp. vect. in \mathbb{F}_q^n}{=} \frac{\log(q^k)}{\log(q^n)} = \frac{k\log(q)}{n\log(q)} = \frac{k}{n}.$$

si observand ca $dim(C) = nR(C)$ avem:

$$dim(C) = n\frac{k}{n} = k$$

care este dimensiunea lui C ca subspatiu vectorial in $\mathbb{F}_q^n$.

Notatie 7.1 (Cod liniar q-ar de lungime n, dimensiune k, si distanta minima d): Daca C este un cod liniar q-ar de lungime n si dimensiune k, spunem ca C este un $[n, k]_q$-cod.

Daca in plus distanta minima a lui C este d, spunem ca C este un $[n, k, d]_q$-cod.

Propozitie 7.2 (Distanta minima a unui cod liniar): *Daca $C \leq_{\mathbb{F}_q} \mathbb{F}_q^n$ este un cod liniar, atunci:*

$$\Delta(C) = \min\{wt(v) | v \in C\}$$

Justificare: Fie $v \in C$ cu $wt(v) = \min\{wt(v)|v \in C\}$. Ponderea Hamming a lui v este egala cu distanta Hamming dintre v si 0, i.e.:

$$wt(v) = \Delta(v, 0)$$

, iar cum C e subspatiu vectorial avem ca $0 \in C$ si deci distanta minima a lui C nu poate fi mai mare decat $wt(v)$.

Pentru a vedea ca nu poate fi mai mica, observam mai intai ca $\Delta(c_1, c_2) = wt(c_2 - c_1)$, $\forall c_1, c_2 \in C$. Presupunem ca exista $w_1, w_2 \in C$ cu $\Delta(w_1, w_2) < wt(v)$. Cum C e subspatiu vectorial, avem $w_2 - w_1 \in C$, iar cu observatia avem $\Delta(w_1, w_2) \overset{observatia}{=} wt(w_2 - w_1) \overset{presupunerea}{<} wt(v)$. Dar am pornit cu $wt(v) = \min\{wt(v)|v \in C\}$, contradictie.

Definitie 8 (Matrice generator a unui Cod liniar): *Fie $C \leq_{\mathbb{F}_q} \mathbb{F}_q^n$ cod liniar.*

Numim matrice generator a lui C o matrice $G \in \mathcal{M}_{n,k}(\mathbb{F}_q)$ pentru care spatiul generat de coloanele sale (care este subspatiu vectorial in $\mathbb{F}_q^n$) este C, i.e.

$$\langle\{col_1(G), ..., col_k(G)\}\rangle = C$$

.

Observam ca G nu este neaparat unica. C are atatea matrici generator cat are baze (privite ca multimi ordonate).

Propozitie 9 (Forma sistematica): *Fie C un cod liniar de lungime n si dimensiune k. Spunem ca o matrice generator G' este in forma sistematica sau forma standard daca:*

$$G' = \begin{pmatrix} I_k \\ P \end{pmatrix}$$

unde $P \in \mathcal{M}_{(n-k)\times k}(\mathbb{F}_q)$. Orice matrice generator G poate fi adusa la forma sistematica prin operatii elementare pe coloane.

Daca o matrice generator, adica matricea asociata functiei de codificare, este in forma sistematica, atunci decodificarea este usoara deoarece primele k componente ale mesajului codificat sunt chiar mesajul original.

Justificare: A face operatii elementare pe linii si coloane asupra matricei G inseamna inmultirea acesteia la stanga sau la dreapta cu matrici de forma:

$$T_{i,j}(a) \overset{def}{:=} \begin{pmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & 1 & \dots & a & & \\ & & \vdots & \ddots & \vdots & & \\ & & 0 & \dots & 1 & & \\ & & & & & \ddots & \\ & & & & & & 1 \end{pmatrix} \quad P_{i,j} \overset{def}{:=} \begin{pmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & 0 & \dots & 1 & & \\ & & \vdots & \ddots & \vdots & & \\ & & 1 & \dots & 0 & & \\ & & & & & \ddots & \\ & & & & & & 1 \end{pmatrix}$$

$$E_i(a) \overset{def}{:=} \begin{pmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & 1 & & & & \\ & & & a & & & \\ & & & & 1 & & \\ & & & & & \ddots & \\ & & & & & & 1 \end{pmatrix}$$

unde consideram $T_{i,j}(a)$, $P_{i,j}$, $E_i(a) \in \mathcal{M}_{n\times n}(\mathbb{F}_q)$ daca inmultim cu ele la stanga sau $\mathcal{M}_{k\times k}(\mathbb{F}_q)$ daca inmultim cu ele la dreapta.

Inmultirea la stanga si la dreapta corespund operatiilor pe linii, respectiv pe coloane, astfel: inmultirea la dreapta cu $T_{i,j}(a)$ corespunde operatiei "$col_i(G) := col_i(G) + acol_j(G)$", cu $P_{i,j}$ corespunde *interschimbarii coloanelor i si j* , iar cu $E_i(a)$ corespunde operatiei "$col_i(G) := acol_i(G)$ " (analog pentru inmultirea la dreapta si operatii pe linii).

Aceste matrici sunt inversabile, deci inmultirea cu ele nu modifica rangul matricei G.

Daca facem doar operatii pe coloane, atunci spatiul generat de coloanele matricei obtinute ramane acelasi ca cel al matricei G, adica C.

Prin inmultirea la dreapta cu astfel de matrici, adica prin operatii elementare pe coloanele matricei G, putem obtine o matrice de forma cautata: $\begin{pmatrix} I_k \\ P \end{pmatrix} \overset{not.}{=:} G'$ unde $P \in \mathcal{M}_{(n-k)\times k}(\mathbb{F}_q)$, iar $\langle\{col_i(G')|i \in \{1, ..., k\}\}\rangle = \langle\{col_i(G)|i \in \{1, ..., k\}\}\rangle = C$.

3.1.3 Matrici de Parity check

Definitie 10 (Matrice de Parity check): *Fie $C \subseteq \mathbb{F}_q^n$. Atunci C este cod liniar daca si numai daca exista o matrice $H \in \mathcal{M}_{(n-k)\times n}(\mathbb{F}_q)$ de rang maxim $n-k$ astfel incat:*

$$C = \{v \in \mathbb{F}_q^n | Hv = 0\}$$

adica C este kernelul transformarii liniare asociate matricei H.

O astfel de matrice H se numeste matrice de parity check.

Justificare: " $\Longleftarrow$ ": Fie $T : \mathbb{F}_q^n \to \mathbb{F}_q^{n-k}$ transformarea liniara asociata matricei H. Din Teorema Fundamentala de Izomorfism pentru spatii vectoriale, avem ca daca T este transformare liniara, atunci $Ker(T)$ este subspatiu vectorial in $\mathbb{F}_q^n$. Dar $Ker(T) = C$, ceea ce e ce voiam.

" $\Longrightarrow$ ": C este cod liniar, deci admite o matrice generator G. Prin operatii elementare pe coloane, putem aduce matricea G la forma $\begin{pmatrix} I_k \\ P \end{pmatrix} \overset{not.}{=:} G'$ unde $P \in \mathcal{M}_{(n-k)\times k}$, iar spatiul generat de coloanele lui G' este acelasi cu spatiul generat de coloanele lui G, adica C.

Vrem sa vedem ca exista o matrice $H \in \mathcal{M}_{(n-k)\times n}$ astfel incat $Hv = 0, \forall v \in C$. Pentru aceasta este suficient ca $Hv = 0, \forall v \in B$ unde B este o baza a lui C. Coloanele matricei G, dar si coloanele matricei G' sunt baze pentru C. Deci e suficient sa avem

$$HG' = O_{(n-k)\times k}$$

ceea ce se verifica daca punem $H := \begin{pmatrix} -P & I_{n-k} \end{pmatrix}$.

Observatie: Daca C este cod binar, atunci $-P = P$, deoarece fiecare element al $\mathbb{F}_2$ este propriul sau invers aditiv.

Propozitie 11 (Matrici de Parity check si Distanta minima a unui Cod): *Fie $C \leq_{\mathbb{F}_q} \mathbb{F}_q^n$ cod liniar, si H matrice de parity check pentru C. Atunci $\Delta(C)$ este egala cu numarul minim de coloane liniar dependente ale matricei H.*

3.1.4 Codul Hamming

Definitie 12 (Codul Hamming $[7,4,3]_2$): *Definim codul Hamming $[7,4,3]_2$ (si notam C_{Ham}) prin functia sa de codificare $T : \mathbb{F}_2^4 \to \mathbb{F}_2^7$ definita prin:*

$$T\left(\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}\right) \overset{def.}{:=} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_2 + x_3 + x_4 \\ x_1 + x_3 + x_4 \\ x_1 + x_2 + x_4 \end{pmatrix}$$

care este o transformare liniara cu o matrice generator G in forma sistematica:

$$G \overset{not.}{:=} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_2 + x_3 + x_4 \\ x_1 + x_3 + x_4 \\ x_1 + x_2 + x_4 \end{pmatrix}$$

Codul Hamming are distanta minima 3, deci poate fi folosit pentru a corecta erori de o componenta. Exemplificam cum se poate face acest lucru:

Matricea definita prin:

$$H \overset{not.}{:=} \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

verifica $HG = O_{(7-4)\times 4}$ deci este o matrice de parity check pentru C_{Ham}.

Observam ca coloanele matricii H sunt reprezentarea binara a numerelor de la 1 la 7.

Consideram un codeword care a suferit un bit-flip: $y := c + \varepsilon$ unde $c, \varepsilon \in C$ cu $wt(\varepsilon) = 1$. Atunci avem:

$$Hy = H(c+\varepsilon) \overset{\text{H liniara}}{=} Hc + H\varepsilon \overset{\text{H parity check}}{=} 0 + H\varepsilon = H\varepsilon$$

iar cum $wt(\varepsilon) = 1$, ε este e_i un vector din baza canonica pentru un anume i, deci:

$$H\varepsilon = He_i = col_i(H)$$

care din definitia lui H este reprezentarea binara a lui i, si astfel am gasit pozitia i a bit-ului care a fost flipped.

Definitie 13 (Syndrome): *Vectorul Hy ca mai sus se numeste syndrome-ul lui y.*

Definitie 14 (Codul Hamming generalizat $[2^r - 1, 2^r - 1 - r, 3]_2$): *Fie $H_r \in \mathcal{M}_{r\times(2^r-1)}(\mathbb{F}_2)$ matricea definita prin:*

$$col_i(H_r) \overset{def.}{:=} \textit{reprezentarea binara a lui } i$$

Deci coloanele $col_j(H_r)$ unde $j \in \{2^0, 2^1, ..., 2^{r-1}\}$ vor fi vectorii bazei canonice $e_1, ..., e_r \in \mathbb{F}_2^r$, deci rangul lui H_r este maxim. Definim al r-lea cod Hamming generalizat $C_{Ham}^{(r)}$ ca fiind codul liniar pentru care H_r este matrice de parity check, adica:

$$C_{Ham}^{(r)} \overset{def.}{:=} \{v \in \mathbb{F}_2^{2^r-1} | H_r v = 0\}$$

Rangul matricei H_r este r, deci:

$$n - k = r \iff k = n - r = 2^r - 1 - r$$

deci dimensiunea codului C_{Ham}^r este:

$$\dim(C_{\text{Ham}}^r) = 2^r - 1 - r$$

Cum coloanele $1, 2$ si 3 sunt liniar independente iar orice doua coloane sunt liniar independente (pentru ca sunt distincte), din **Propozitia 11** avem ca

$$\Delta(C_{\text{Ham}}^{(r)}) = 3$$

3.1.5 Dualul unui Cod liniar

Definitie 15 (Dualul unui cod liniar): *Fie C cod liniar. Definim codul dual al lui C si notam $C^{\perp}$ca fiind subspatiul vectorial ortogonal pe acesta, adica:*

$$C^{\perp} \overset{def.}{:=} \{v \in \mathbb{F}_q^n | \langle v, c \rangle = 0, \forall c \in C\}$$

unde $\langle v, c \rangle \overset{def.}{:=} \Sigma v_i x_i$ este produsul scalar pe $\mathbb{F}_q^n$.

Propozitie 16 (Proprietati ale Dualului): *(1) Daca C este un $[n,k]_q$-cod, atunci $C^{\perp}$ este un $[n, n-k]_q$-cod*

(2) $(C^{\perp})^{\perp} = C$

(3) Daca H este o matrice parity check pentru C atunci H^T este o matrice generator pentru $C^{\perp}$. Echivalent, daca G este o matrice generator pentru C, atunci G^T este o matrice de parity check pentru $C^{\perp}$.

Observatie: Spre deosebire de $\mathbb{R}$-spatiile vectoriale unde daca $S \leq_{\mathbb{R}} \mathbb{R}^n$ atunci $\mathbb{R}^n = S \bigoplus S^{\perp}$, pentru $\mathbb{F}_q$-spatii vectoriale acest lucru nu mai este adevarat. Se poate de exemplu, ca $C^{\perp} \subseteq C$ (spunem ca C este *auto-ortogonal*) sau chiar $C^{\perp} = C$ (spunem ca C este *auto-dual*).

De exemplu:

$$C := \{\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix}\} \leq_{\mathbb{F}_2} \mathbb{F}_2^2$$

este auto-ortogonal.

3.1.6 Codul Hadamard

Codul dual al C_{Ham^r} are ca matrice generator $G_r := H_r^T \in \mathcal{M}_{(2^r-1)\times r}(\mathbb{F}_2)$. Acesta este un $[2^r-1, r]_2$-cod si se numeste *codul simplex*. *Codul Hadamard* se obtine adaugand linia $(0, ..., 0)$ la matricea G_r, adica:

$$G_{\mathrm{Had}} := \begin{pmatrix} 0...0 \\ G_r \end{pmatrix}$$

Definitie 17 (Codul Hadamard $[2^r, r, 2^r-1]_2$): *Codul Hadamard (notat Had_r) are ca matrice generator matricea G_{Had} ale carei linii sunt reprezentarile binare ale numerelor de la 0 la 2^r-1, adica toate elementele lui $\mathbb{F}_q^{2^r}$.*

4 Algoritm si implementare

Programele descrise mai jos se gasesc pe `github` sub forma de fisiere `.py` astfel:

Programul 1 – `Hadamard_generalizat.py`

Programul 2 – `Hamming_7_4_3.py` si `Hamming_15_11_3.py` s

Programul 1 (Corectare de erori cu Codul Hadamard generalizat): Primul program este o exemplificare faptului ca, spre deosebire de codul Hamming generalizat, codul Hadamard generalizat $[2^r, r, 2^{r-1}]$ poate corecta mai mult de 1 bit-flip (maxim $\frac{2^{r-1}-1}{2}$ bit-flip-uri).

Implementarea este astfel:

Metoda pentru a genera matricea generator a codului Hadamard generalizat $G_{\mathbf{Had}}$. Metoda `generator_matrix(r)` primeste ca input numarul r, apoi se uita la numerele $x \in \{0, 1, ..., 2^{r-1}\}$ si obtine scrierea lor in baza 2 facand, pentru fiecare x, operatia de bitwise-and `&` intre x si fiecare $m \in \{2^0, 2^1, ..., 2^{r-1}\}$. Avand aceste scrieri in baza 2, construieste matricea G_{Had} conform definitiei ei.

Metoda pentru a simula transmiterea mesajului codificat printr-un canal de comunicatie zgoomotos. Metoda `add_noise(v, nr_erori)` primeste ca input un vector $v \in \mathbb{F}_2^{2^r}$ si returneaza $v + \varepsilon$ unde $\varepsilon \in \mathbb{F}_2^{2^r}$ este generat aleator folosind `numpy.random.default_rng.choice` (pentru a permite extragerea fara intoarcere adica generarea de numere distincte, pentru a asigura ca $wt(\varepsilon)$ va fi cat l-am ales) si putem alege cat sa fie $wt(\varepsilon)$ adica numarul de bit-flip-uri pe care vrem sa le introducem.

Metoda principala. Metoda principala `error_correct(r, nr_erori)`. Alegem un r. Apoi generam aleator, folosind `numpy.random.randint`, un vector de lungime r ce reprezinta un mesaj $m \in \mathbb{F}_2^r$. Codificam acest mesaj cu codul Hadamard generalizat prin inmultirea cu matricea G_{Had}:

```
G_Had = generator_matrix(r)
```

```
m_encoded = G_Had @ m
```

dupa care introducem zgomot cu un numar de erori ales $\leq \frac{2^{r-1}-1}{2}$:

```
m_encoded_zgomotos = add_noise(m_encoded, nr_erori_ales)
```

Mai departe, in baza **Propozitiei 6**, stim ca exista un unic vector $v \in G_{\text{Had}}\mathbb{F}_2^r$ astfel incat $\Delta(v, \texttt{m_encoded_zgomotos}) < \frac{2^{r-1}-1}{2}$, acela fiind mesajul original `m`. Pentru a-l gasi, vom considera matricea ale carei coloane sunt toate elementele multimii $G_{\text{Had}}\mathbb{F}_2^r$, care este:

```
G_Had @ G_Had.transpose()
```

vom scadea `m_encoded_zgomotos` din fiecare coloana a acesteia, apoi vom calcula ponderea Hamming a fiecarei coloane ($= \Delta(\texttt{m_encoded_zgomotos}$, coloana respectiva), iar coloana cu ponderea minima este mesajul codificat original `m_encoded`.

Programul 2 (Codificare, corectarea unui singur bit-flip si decodificare cu Codul Hamming): Al doilea program este un exemplu de codificare, corectare a unui singur bit-flip si decodificare folosind mai intai codul Hamming $[7, 4, 3]_2$ si apoi codul Hamming $[15, 11, 3]_2$. Mesajul $m \in \mathbb{F}_2^4$ si vectorul zgomot $\varepsilon \in \mathbb{F}_2^7$ cu $wt(\varepsilon) = 1$ sunt generate aleator folosind `numpy.random.randint()`. Algoritmul este cel descris in **Sectiunea 2 – Definitia 12**.

5 Experimente

5.1 Plot-uri cu performanta

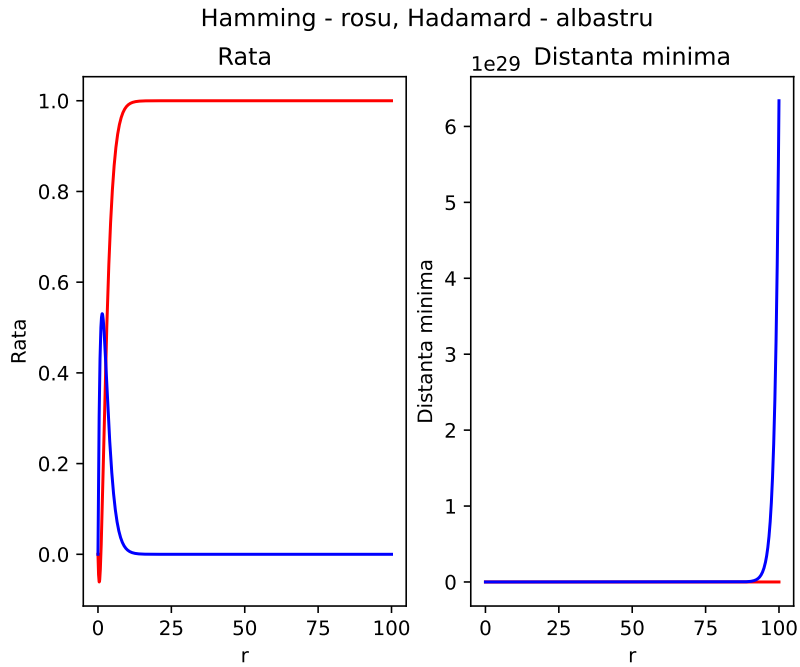

Figure 1: Grafic al ratei si distantei minime ale celor 2 coduri in functie de r.

Graficul 1 (Rata si Distanta minima a codurilor Hamming $[2^r - 1, 2^r - 1 - r, 3]_2$ **si Hadamard** $[2^r, r, 2^{r-1}]$**):** Graficul ilustreaza comportamentul asimptotic al ratei si distantei minime ale celor doua coduri generalizate, cand $r \longrightarrow \infty$. Observam ca:

$$R(C_{\text{Ham}}^{(r)}) \underset{r\to\infty}{\longrightarrow} 1$$

adica redundanta tinde la 0 cand $r \longrightarrow \infty$, lucru de dorit, in timp ce:

$$R(\text{Had}_r) \underset{r\to\infty}{\longrightarrow} 0$$

lucru care nu e de dorit.

In schimb, situatia este invers cand vine vorba de distanta minima. Avem:

$$\Delta(C_{\text{Ham}}^{(r)}) = 3, \forall r \in \mathbb{N}$$

adica nu poate corecta decat maxim un bit-flip si poate detecta maxim doua, in timp ce:

$$\Delta(\text{Had}_r) \underset{r\to\infty}{\longrightarrow} \infty$$

adica oricate erori am fi nevoiti sa corectam sau detectam intr-un mesaj transmis, exista un $r \in \mathbb{N}$ pe care il putem alege a.i. Had_r sa poata corecta sau detecta acel numar de erori.

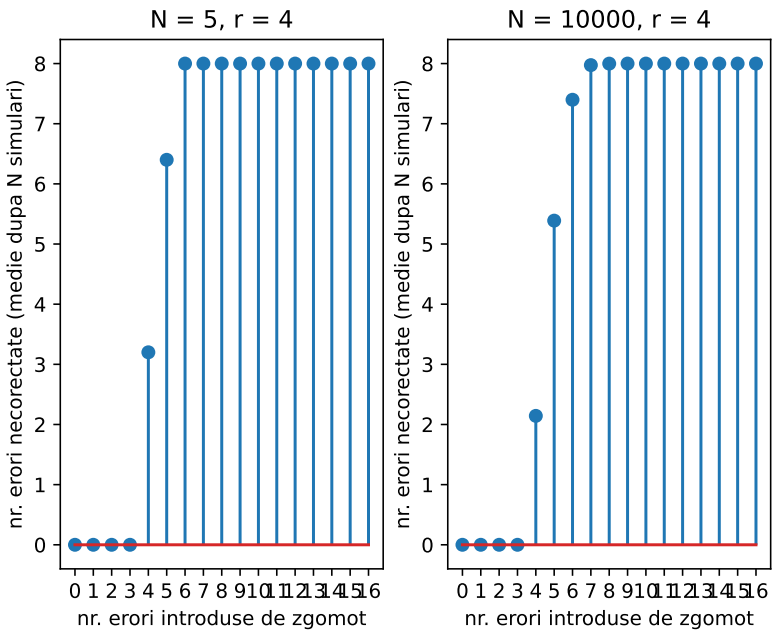

Figure 2: Numarul de erori ramase necorectate (in medie, dupa N simulari) in functie de numarul de erori introduse prin zgomot

Graficul 2 (Numarul de erori ramase necorectate (in medie, dupa N simulari) in functie de numarul de erori introduse prin zgomot, pentru codul Hadamard): Transmitem N mesaje codificate folosind Had_r prin un canal zgomotos de comunicatie, care introduce un zgomot $\varepsilon \in \mathbb{F}_2^{2^r}$. Ne uitam la media numarului de erori care au ramas necorectate, pentru fiecare $wt(\varepsilon)$ posibil, adica $\forall wt(\varepsilon) \in \{0, 1, ..., 2^r\}$. Graficul are pe axa orizontala $wt(\varepsilon)$ si pe axa verticala media numarului de erori dupa N simulari.

6 Concluzii si future work

6.1 Cateva concluzii

Codul Hamming generalizat are o rata ce converge la valoarea optima 1, deci cu cat r este mai mare cu atat codeword-urile sunt mai putin redundante. Insa

distanta sa minima este constant egala cu 3, deci nu poate corecta mai mult de 1 bit-flip si detecta mai mult de doua.

In contrast, codul Hadamard generalizat are o rata ce converge la 0, adica cu cat r creste cu atat codeword-urile devin mai redundante, "la limita necontinand nici o informatie folositoare". Insa distanta sa minima tinde la infinit, deci ne permite sa corectam oricat de multe erori, daca acceptam un r suficient de mare.

6.2 Future work

Studiul altor coduri liniare, precum codurile BCH sau codurile Reed-Solomon, ce folosesc alte particularitati matematice pentru a asigura corectarea erorilor (interpolare Lagrange, transformata Fourier discreta).

Implementarea unei program in care mesajul transmis este sub forma de sir de caractere ASCII, codificarea, corectarea erorilor si decodificarea sa, atat cu codul Hamming generalizat cat si cu codul Hadamard generalizat.

References

[1] Massoud Malek. "Hadamard Codes". In: (2006). URL: https://web.archive.org/web/20200109044013/http://www.mcs.csueastbay.edu/~malek/TeX/Hadamard.pdf.

[2] "Notes 1: Introduction, linear codes". In: (2010). URL: https://www.cs.cmu.edu/~venkatg/teaching/codingtheory/notes/notes1.pdf.